

Machine Learning for social sciences

Natural Language Processing: Session 3

William Aboucaya

Connecting to Previous Classes

We now build on previous concepts:

- ML basics using regression
- Discrete output handling from classification
- Text tokenization and semantic vectorization
- Structure discovery from clustering
- Semantic reasoning from NLI

Connecting to Previous Classes

We now build on previous concepts:

- ML basics using regression
- Discrete output handling from classification
- Text tokenization and semantic vectorization
- Structure discovery from clustering
- Semantic reasoning from NLI

Today's Focus

How models generate text and how to adapt them for a given task.

What Does a Language Model Learn?

A language model captures patterns in text:

Core Idea

It estimates $p(w_t \mid w_1, \dots, w_{t-1})$, the probability of the next token.

Examples of what it implicitly learns:

- Syntax: word order, agreement, punctuation
- Semantics: meaning, relations between concepts
- World knowledge: common facts and associations

Tokenization Beyond Basics

Tokenization converts text into model-friendly units:

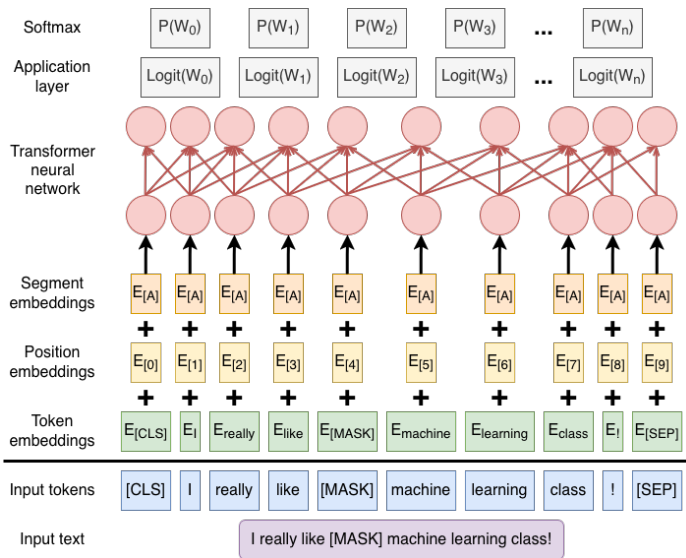
- Subwords reduce vocabulary size
- Rare words decomposed into frequent pieces

Popular tokenizers: BPE, WordPiece, SentencePiece.

Example

“unbelievable” → “un”, “##believ”, “##able”

How Generation Actually Happens



Deterministic:

- Greedy: chooses highest-probability token
- Beam search: explores multiple candidate continuations

Deterministic:

- Greedy: chooses highest-probability token
- Beam search: explores multiple candidate continuations

Stochastic:

- Temperature scaling
- Top-k and nucleus sampling

Deterministic:

- Greedy: chooses highest-probability token
- Beam search: explores multiple candidate continuations

Stochastic:

- Temperature scaling
- Top-k and nucleus sampling

Tradeoff

Deterministic methods improve consistency; sampling improves creativity.

Greedy Search

Definition: A greedy search (or hill-climbing) algorithm is an algorithm that always takes the best *immediate*, or local, solution while finding an answer¹

When applied to text generation, a greedy search algorithm will always choose the most probable next token, not looking at any future iteration step.

¹Paul E. Black. *Greedy algorithm*. 2005.

Greedy Search

Definition: A greedy search (or hill-climbing) algorithm is an algorithm that always takes the best *immediate*, or local, solution while finding an answer¹

When applied to text generation, a greedy search algorithm will always choose the most probable next token, not looking at any future iteration step.

Pros:

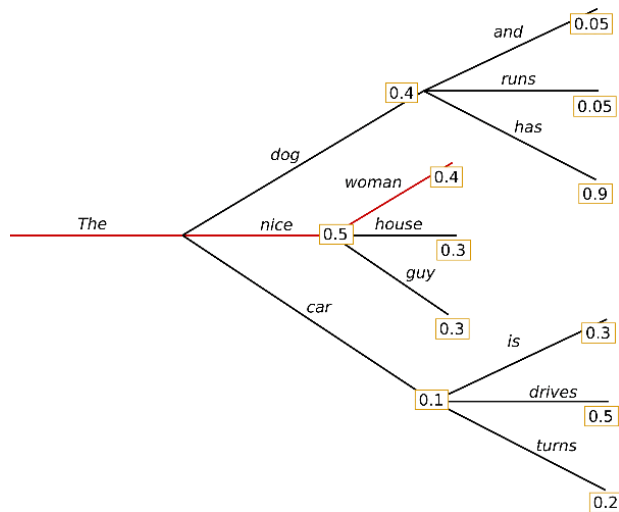
- Extremely easy to implement
- Every token prediction computed will be used in the final result → very effective

Cons:

- Tends to find local optimums (by design)
- Infinite token loops

¹Paul E. Black. *Greedy algorithm*. 2005.

Greedy Search



Example from <https://huggingface.co/blog/how-to-generate>

Definition: A beam search algorithm tries to find the optimal solution in a tree search task by memorizing at each step the β best solutions and continuing the exploration from these best states at the next exploration step. β is called the beam width and is a parameter to choose.

When applied to text generation, a beam search algorithm will follow the β most probable sequences of tokens, developing them until finding the most probable solution among the sequences predicted.

Definition: A beam search algorithm tries to find the optimal solution in a tree search task by memorizing at each step the β best solutions and continuing the exploration from these best states at the next exploration step. β is called the beam width and is a parameter to choose.

When applied to text generation, a beam search algorithm will follow the β most probable sequences of tokens, developing them until finding the most probable solution among the sequences predicted.

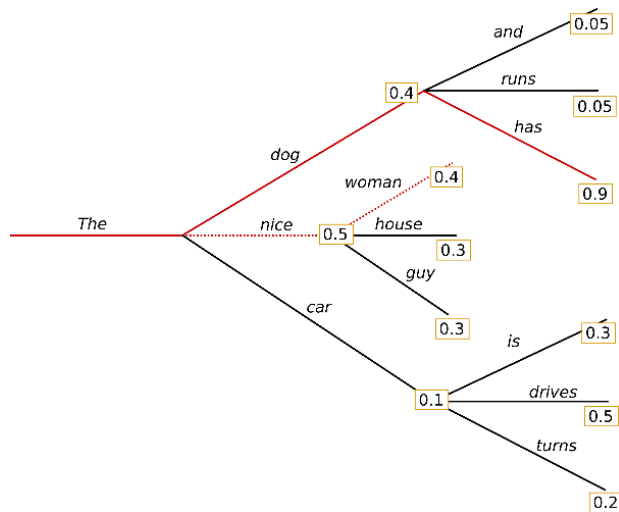
Pros:

- More fluent results than greedy search
- Possibility to propose multiple predictions

Cons:

- For each next-token prediction, predicts β new tokens $\rightarrow$ less effective if β is large
- Still risks to produce infinite token loops

Beam Search



Example from <https://huggingface.co/blog/how-to-generate>

Definition: Given the probability distribution of a next token prediction, randomly select the next token among the most probable tokens based on their probability.

Pre-selection of tokens is either done by selecting the k most probable tokens (**top- k sampling**), or the smallest list of tokens so that their cumulative probability is higher than p (**nucleus sampling**).

Definition: Given the probability distribution of a next token prediction, randomly select the next token among the most probable tokens based on their probability.

Pre-selection of tokens is either done by selecting the k most probable tokens (**top- k sampling**), or the smallest list of tokens so that their cumulative probability is higher than p (**nucleus sampling**).

Pros:

- Unlikely to repeat itself
- Often more fluent than previous methods

Cons:

- Introduces randomness, i.e. not suitable if you need reproducibility

Temperature: A parameter in the *softmax* probability distribution:

$$\text{softmax}(y_i) = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^K e^{\frac{y_k}{T}}}$$

With T the temperature, i a class in the set $\{1, \dots, K\}$

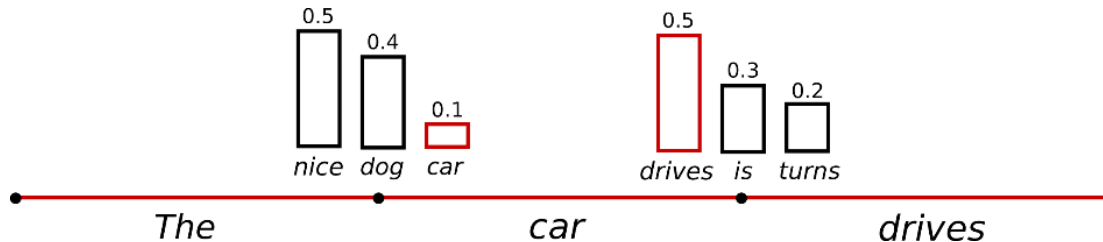
Temperature: A parameter in the *softmax* probability distribution:

$$\text{softmax}(y_i) = \frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^K e^{\frac{y_k}{T}}}$$

With T the temperature, i a class in the set $\{1, \dots, K\}$

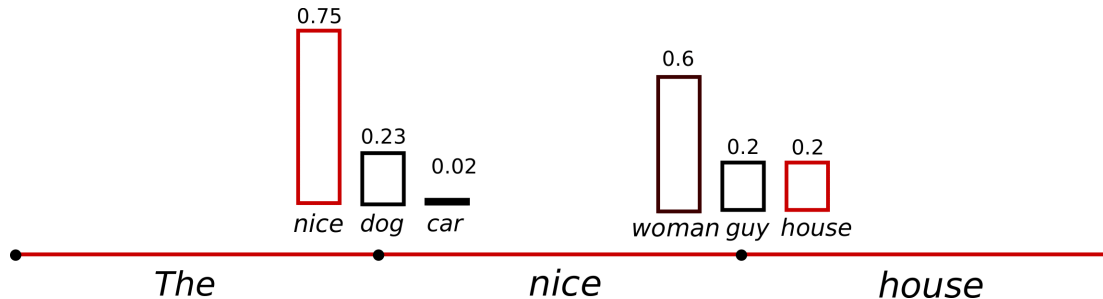
- Higher temperature $\rightarrow$ all classes have similar probability, i.e. more randomness and “creativity”
- Lower temperature $\rightarrow$ further increase the probability of most probable classes, i.e. more stability

With $T = 1$



Example from <https://huggingface.co/blog/how-to-generate>

With $T = 0.6$



Example from <https://huggingface.co/blog/how-to-generate>

- ① Transform pre-existing text into tokens

Summary

- ① Transform pre-existing text into tokens
- ② Produce token embeddings using the first layer of your language model

Summary

- ① Transform pre-existing text into tokens
- ② Produce token embeddings using the first layer of your language model
- ③ Apply the following layers of the neural network to better represent your text as a whole

Summary

- ① Transform pre-existing text into tokens
- ② Produce token embeddings using the first layer of your language model
- ③ Apply the following layers of the neural network to better represent your text as a whole
- ④ Obtain a probability distribution of your next token

Summary

- ➊ Transform pre-existing text into tokens
- ➋ Produce token embeddings using the first layer of your language model
- ➌ Apply the following layers of the neural network to better represent your text as a whole
- ➍ Obtain a probability distribution of your next token
- ➎ Choose your predicted token according to your decoding strategy (greedy search, beam search, sampling)

Summary

- ➊ Transform pre-existing text into tokens
- ➋ Produce token embeddings using the first layer of your language model
- ➌ Apply the following layers of the neural network to better represent your text as a whole
- ➍ Obtain a probability distribution of your next token
- ➎ Choose your predicted token according to your decoding strategy (greedy search, beam search, sampling)
- ➏ Repeat until the model produces a [EOS] token or a predefined number of iteration is reached

Summary

- ➊ Transform pre-existing text into tokens
- ➋ Produce token embeddings using the first layer of your language model
- ➌ Apply the following layers of the neural network to better represent your text as a whole
- ➍ Obtain a probability distribution of your next token
- ➎ Choose your predicted token according to your decoding strategy (greedy search, beam search, sampling)
- ➏ Repeat until the model produces a [EOS] token or a predefined number of iteration is reached

Let's see what we can do with it 🤓

Task-specific text generation

So far, we have asked the model to continue a text.

Now we want to use our model to **apply a task requiring text generation**, where the quality of the text generation can be **measured**.

Here, we will train our model for natural language text simplification.

What Is Text Simplification?

Objective: Making a text easier to read and understand

Depends on 3 main attributes²:

- **Lexical and syntactic complexity:** simpler words and sentence structures
- **Amount of compression:** lower number of words
- **Paraphrasing:** meaning stays unchanged

²Louis Martin et al. "Controllable Sentence Simplification". In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Marseille, France: European Language Resources Association, May 2020.

What Is Text Simplification?

Objective: Making a text easier to read and understand

Depends on 3 main attributes²:

- **Lexical and syntactic complexity:** simpler words and sentence structures
- **Amount of compression:** lower number of words
- **Paraphrasing:** meaning stays unchanged

Example

“The physician administered the medication” → “The doctor gave the medicine”

²Louis Martin et al. “Controllable Sentence Simplification”. In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Marseille, France: European Language Resources Association, May 2020.

Fine-Tuning for text simplification

Fine-tuning can be applied for text generation, but we need a dataset of simple and complex text pairs

Mechanism

Weights are adjusted using supervised pairs (complex $\rightarrow$ simple).

Benefits:

- Transfers broad language understanding
- Learns task-specific rewriting patterns
- Requires relatively small datasets

The Wikilarge dataset

Wikilarge³ is a dataset consisting of pairs of texts from Wikipedia in English (complex) and Wikipedia in simple English (simple).

Example

Complex text: Leventina is one of the eight districts of the largely Italian-speaking canton of Ticino in Switzerland.

Simple text: Leventina is a district of the canton of Ticino in Switzerland.

³Xingxing Zhang and Mirella Lapata. "Sentence Simplification with Deep Reinforcement Learning". In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, Sept. 2017.

Fine-tuning workflow

- 1 Dataset preparation
- 2 Tokenization and formatting
- 3 Model training
- 4 Validation and metric tracking
- 5 Final evaluation

Key Metric

SARI is the preferred metric for text simplification. It measures simplification quality by comparing the generated text to both the input and a series of human translations. It rewards:

- Adding helpful words
- Deleting unnecessary or complex words
- Keeping appropriate words from the original sentence

The hyperparameters that we will use :

- **Learning Rate:** Too high: unstable training. Too low: slow convergence.
- **Batch Size:** Larger batches = smoother gradients, higher memory use.
- **Epochs:** More passes risk overfitting.
- **Warmup:** Steps Helps stabilize early training.
- **Weight Decay:** Regularizes to avoid memorizing examples.

Recommendations:

- Regularly inspect model outputs during fine-tuning.
- Use validation loss + SARI for checkpoint selection.
- Apply early stopping when simplification quality stops improving.
- Re-run with different sets of hyperparameters depending on issues identified during training.

Let's see what we can do with it 🤓